Assignment 2: Comparing Models to Predict Diabetes

Overview

Students train three classifiers — K-nearest neighbours (KNN), a decision tree and logistic regression — on the same diabetes data, compare their test accuracy in a bar chart, and explain which model predicts best. The workflow is the one practised in class on the Iris dataset, now applied to a real health problem.

Item	Detail
Mode	Individual
Tools	Python in Google Colab; pandas, scikit-learn, matplotlib
Data	`diabetes.csv` (768 patients, 8 features, 1 label)
Effort	About 4–5 hours
Submission	One `.ipynb` notebook with answers in markdown cells

Learning outcomes. Students will be able to:

1. Load a CSV dataset and separate features (X) from the target (y).
2. Detect missing values hidden as zeros and replace them with column medians.
3. Split data into training and test sets with `train_test_split`.
4. Train KNN, decision tree and logistic regression models with scikit-learn.
5. Build a scikit-learn pipeline that scales features before a model, and explain which models need scaling.
6. Compare models by test accuracy and explain the result in plain language.

The dataset

The Pima Indians Diabetes dataset records 768 female patients aged 21 or over. The target column `Outcome` is 1 if the patient has diabetes and 0 if not; 268 patients (35%) are diabetic.

Column	Meaning
Pregnancies	Number of times pregnant
Glucose	Blood glucose level (mg/dL)
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skinfold thickness (mm)
Insulin	2-hour serum insulin (µU/mL)
BMI	Body mass index (kg/m²)
DiabetesPedigreeFunction	Family-history score for diabetes

Column	Meaning
Age	Age (years)
Outcome	**Target: 1 = diabetic, 0 = not diabetic**

Unlike Iris, which has three flower species, this is a two-class (binary) problem.

Tasks

Follow the same steps as the Iris model-comparison notebook from class. Use `random_state=42` so your results can be checked.

Task 1 — Load and explore the data

1. Upload `diabetes.csv` to Colab and load it with `pd.read_csv`.
2. Show the first 5 rows and the summary statistics (`df.describe()`).
3. Show the number of patients with each `Outcome` value.

Task 2 — Handle missing values

In this dataset, missing readings were stored as 0. A glucose level, blood pressure, skin thickness, insulin level or BMI of 0 is impossible for a living patient, so these zeros must be treated as missing before training.

1. Count the zeros in each column with `(df == 0).sum()`. Which columns contain zeros that cannot be real? Which column's zeros are genuine (hint: Pregnancies)?
2. Replace the zeros in Glucose, BloodPressure, SkinThickness, Insulin and BMI with `np.nan`, then confirm the counts with `df.isnull().sum()`.
3. Fill each missing value with the median of its column using `df.fillna(df.median())`.
4. In 1–2 sentences, explain why the median is a safer choice than the mean for a skewed column such as Insulin.
5. Run `df.describe()` again and confirm that no impossible zeros remain.

Hint for step 2:

```
import numpy as np
cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[cols] = df[cols].replace(0, np.nan)
```

Task 3 — Split the data and train three models

Using the **cleaned** data, set `X` to the 8 feature columns and `y` to `Outcome`. Split into 80% training and 20% test data with `stratify=y` and print the size of each set. Then train each model on the training set only, predict on the test set, and print its accuracy:

Model	Settings to use
K-Nearest Neighbors	`KNeighborsClassifier(n_neighbors=3)`
Decision Tree	`DecisionTreeClassifier(random_state=42)`

Model	Settings to use
Logistic Regression	`LogisticRegression(max_iter=1000, random_state=42)`

Use a dictionary and a `for` loop as in class. Starter code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv("diabetes.csv")

# Task 2: handle missing values
cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[cols] = df[cols].replace(0, np.nan)
df = # TODO: fill missing values with each column's median

# Task 3: split and train
X = # TODO: the 8 feature columns
y = # TODO: the Outcome column

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Task 4 — Scale the features with a pipeline

The features sit on very different ranges: Insulin goes up to 846 while DiabetesPedigreeFunction stays below 2.5. KNN measures the distance between patients, so large-valued columns dominate that distance unless the features are scaled.

1. For each of the three models, build a pipeline that standardises the features and then applies the model, using `make_pipeline(StandardScaler(), model)`. Keep the same settings as in Task 3.
2. Fit each pipeline on the training set, predict on the test set and print its accuracy.
3. Make a table of each model's accuracy without scaling (Task 3) and with scaling.
4. In 2–3 sentences, explain why scaling changes KNN's accuracy but leaves the decision tree's unchanged.
5. In one sentence, explain why the scaler inside the pipeline learns its mean and standard deviation from the training data only, and what that prevents.

Starter code:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
pipelines = {
    "KNN": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=3)),
    "Decision Tree": make_pipeline(StandardScaler(),
DecisionTreeClassifier(random_state=42)),
    "Logistic Regression": make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000, random_state=42)),
}

scaled_accuracies = {}
for name, pipe in pipelines.items():
    # TODO: fit the pipeline on X_train, predict on X_test,
    # and store the accuracy (%) in scaled_accuracies
    pass
```

Task 5 — Compare the models

1. Draw a grouped bar chart of all six test accuracies: each model without scaling (Task 3) and with scaling (Task 4), with a percentage label on each bar.
2. Set the y-axis range so all bars are visible.
3. Print a `classification_report` for your most accurate model.

Task 6 — Reflect

Answer each question in a markdown cell of 2–4 sentences:

1. Which model was most accurate on the test set, and by how much?
2. The models scored close to 100% on Iris. Why are they much less accurate here?
3. 65% of patients are not diabetic. What accuracy would a model get by always predicting "not diabetic"? Are your models much better than that?
4. Change `n_neighbors` to 5, 9 and 15. How does the accuracy of your scaled KNN pipeline change?
5. A doctor wants to use one model for screening. Which would you choose, and why?
6. Re-run your three models on the original data without cleaning. Did handling the missing values change the accuracies? Suggest a reason.

Submission and marking

Submit one notebook named `StudentID_diabetes.ipynb`. It must run from top to bottom without errors, with all outputs visible.

Due Date: 3 November 2026